ANKARA SCIENCE UNIVERSITY

2025–2026 / SPRING

SENG 324 / Software Design Patterns

TERM PROJECT

Smart Travel Planner Platform

Team-Based Extended Version

You are given a detailed problem description for a software application that involves the use of multiple design patterns. Examine the description carefully and implement the requested application in Java as a **team project**. This version is an improved version of the individual (single-student) assignment, that is extended into a richer collaborative project in which multiple students can own different subsystems and later integrate them into one coherent application.

General Instructions

1. This project is to be completed by a team of **2–3 students**.
2. You should submit:
   - the full source code as a zipped archive,
   - a packaged application jar file (fat jar) that can be executed using `java -jar YourPackagedApp.jar`,
   - a UML class diagram describing the full set of patterns and other key classes in the application,
   - a short contribution report indicating which student implemented which subsystem,
   - a short user manual or README describing how to run and use the application.
3. Your UML diagram should be submitted as a PDF or image file.
4. You may use the provided JSON file containing city entities as the initial input to your application.
5. The mock-up shown in Figure 1 is only illustrative. Your application should provide comparable functionality, but you are free to make the interface more elegant, realistic, and feature-rich.

Problem Description

You are developing a software platform for exploring, monitoring, and planning trips across different cities. The system starts from a list of `City` objects. Each `City` object has the following attributes:

- `name`
- `population`

- `area`
- `currentTemperature`
- `currentWeatherState`

The `currentWeatherState` attribute may take one of the following values: `SUNNY`, `CLOUDY`, `RAINY`, `SNOWY`.

You will be provided with a list of cities in a JSON file. Use this city list as an input to your application.

Base Functionality

Your software should support the following base features:

- A **Singleton** city repository that reads the JSON file once and provides the shared list of cities. In other words the singleton class should initialize itself with a list of cities reading a json file containing all the cities, and should make this list available to requesters from a single point.
- You should design a Graphical User Interface that displays the list of cities in a list box in different order depending on a sorting criteria selected by the user via a combo box. The combo box should present 3 sorting options: *sort by name*, *sort by population*, *sort by area*. Therefore, you should implement an extensible framework that uses the **Strategy Design Pattern**. As such, your software will include various sorting algorithms implemented as sort strategies.
- The user interface should include another list box which should only list cities obeying a given weather condition such as *rainy, sunny, cloudy* or *snowy*. These criteria should be selectable from a combo box, and once selected the city list should change dynamically to reflect the selected criteria. You should implement this behaviour using **Iterator Design Pattern**. So you should have a 4 different iterators each for a different weather condition.
- The user interface should include two charts: one for displaying the temperatures of all cities as a bar chart, and another one displaying a pie chart for percentage of cities with different weather conditions. In other words the pie chart should contain the percentage of sunny, cloudy, rainy and snowy cities. Both of these charts should change dynamically upon receiving updates from a weather report provider object. This weather report provider object should be running on a separate thread and should update the weather information of cities randomly every 3 seconds. You should implement this behaviour using the **Observer Design Pattern** (also known as the **Publisher-Subscriber Design Pattern**).
- Finally you should add a city activity planner to the application to plan activities such as visiting museums, shopping malls, parks, historic city center etc. You don't want to modify the City class directly. You wrap a City into new decorated versions offering extra "visit planning" features. So you should implement this functionality using the **Decorator Design Pattern**. You should add at least 4 decorators (*MuseumVisit, ShoppingMallVisit, ParkVisit* and *CityCenterVisit*). Each decorator should have its own cost and required time in hours. The planner should be available to the user via the GUI as a small pane

where each activity can be added by clicking a check box when a city is selected from the sorted city list.

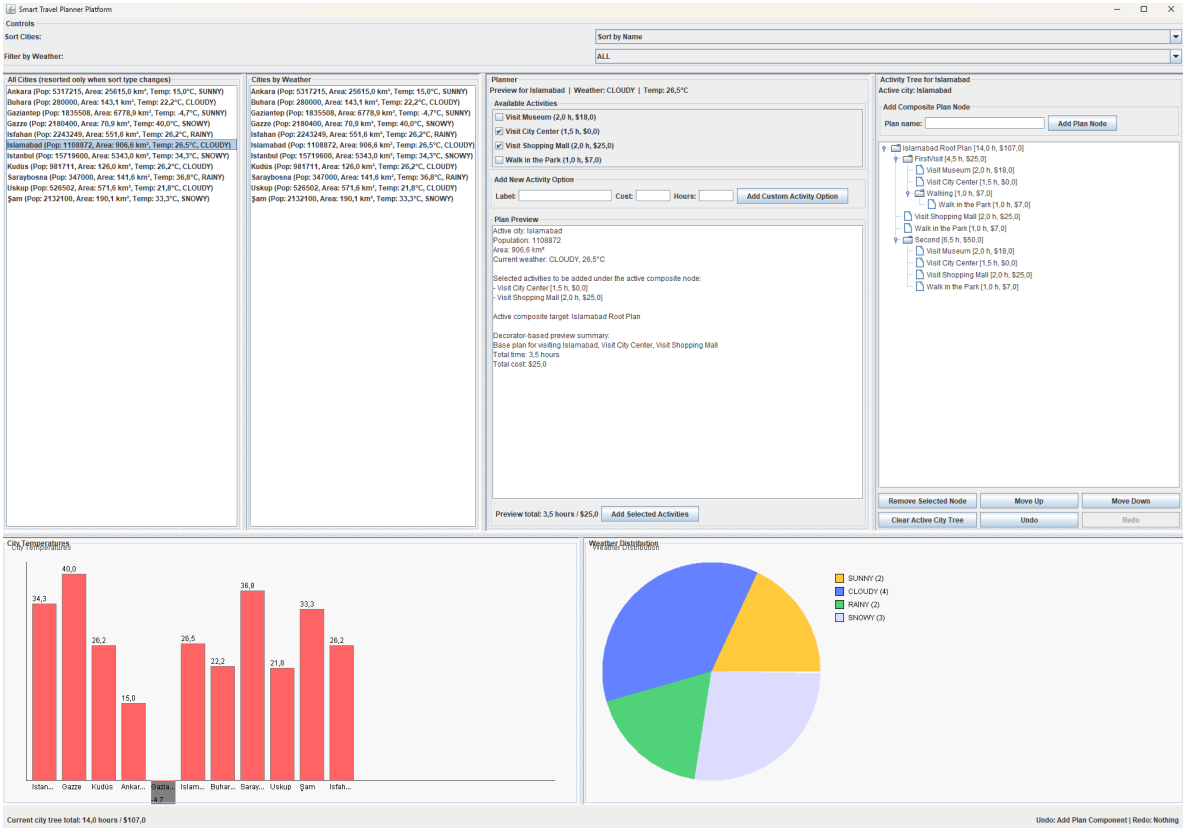

Figure 1: Example GUI mock-up. You may reuse a similar layout or improve it.

Extended Team-Project Requirements

In this extended version, the planner is no longer limited to adding a few activities to a single city. Instead, the user should be able to build a **full travel plan** containing multiple cities, visits, and activities.

Travel Plan Hierarchy The user should be able to create a full plan for a particular city such as:

- One `Main Activity Plan (i.e. a root Activity Plan Node)`
- containing multiple `Activity` entries or other `Activity Plan` Nodes (recursively) each containing one or more activities.

You should implement this functionality using the **Composite Pattern** where `ActivityPlan` represents a **Composite** and `Activity` represents **Leaf (i.e.Component)**

City plans should be built as composites containing activity leaves, and the tree is recursively constructed from these objects. The activity catalog should be extensible from the GUI, so users can add custom activities like cinema or dinner. This hierarchical structure should support:

- recursive listing of all trip components,
- total cost calculation,
- total required time calculation,
- optional removal of a city visit or a single activity from the plan.

Adding composite nodes (Activity Plans) and adding leaf nodes (activities) under a composite node should be done via different buttons. A composite node should only be a `"ActivityPlan"`. All selected activities, such as *visit museum* or *walk in the park* etc. should be added under the active (selected) composite node. If no composite node is active (selected), then the activities can be added to the root node (no duplicates). The relation between a city and activities should be as follows:

- A full activity plan tree will be associated with an active (selected) city.
- When the same city is active (i.e. selected), all the additions (both plans (composites) and activities) will be added to the tree of that particular city.
- If another city is selected and no tree exists for that city then a new tree will be created (with a new root) etc.

Undoable GUI Actions The system should also support richer user interaction. At minimum, the following actions should be available through the GUI:

- add city to trip,
- remove city from trip,
- add activity to selected city visit,
- remove activity from selected city visit,
- move an activity up or down in the displayed plan,
- clear the full plan,
- undo the most recent action,
- redo the previously undone action.

You should implement these user actions using the **Command Pattern**. The GUI should trigger command objects rather than manipulating the model directly.

Expected GUI Features

The GUI should include at least the following panes or panels:

- a control area for sorting and weather filtering,
- a list of all cities,

- a weather-filtered city list,
- a planner panel for adding activities,
- a travel plan panel showing the hierarchical trip structure,
- a bar chart for temperatures,
- a pie chart for weather distribution,
- buttons for undo and redo.

A student team may choose to add additional features such as saving/loading a trip plan, displaying total budget and total duration in a separate panel, or offering search functionality.

Required Design Patterns

Your implementation must meaningfully employ the following design patterns:

1. Singleton
2. Strategy
3. Iterator
4. Observer
5. Decorator
6. Composite
7. Command

Submission Notes

Please provide a complete UML diagram representing all the design patterns involved in this problem. Your UML should clearly show the mapping between pattern roles and problem-specific classes.

Teams will be evaluated not only on correctness, but also on:

- design quality,
- clarity of UML,
- code organization,
- successful pattern usage,
- GUI functionality,
- integration quality across team modules.